
IA para el Desarrollo de Juegos

Bloque 2 — IA Estrategica y Tactica



Autores:

Mohammed Amrou Labied Nasser 49857930W
Sergio Garcia Garcia 49308323A
Diego Sergio Ballesta Gonzalez de la Higuera 30299322L

Tutor:

Francisco Javier Marin-Blazquez Gomez

Grupo / subgrupo:

1.1

Fecha de entrega:

16 de mayo de 2026

Índice

1	Introducción	1
2	Escenario	1
2.1	Descripción	1
2.2	Puntos seguros	2
2.3	Respawn de los NPCs	3
2.4	Objetivos	3
3	Sistema de Combate	4
3.1	Tipo de Unidades y atributos	4
3.1.1	Tabla de ventajas (FDA, FTA, FTD)	4
3.1.2	Daño crítico y bonus por situación	5
3.2	Inicio de combate	6
4	Condiciones de Victoria	7
5	Mapas Tacticos	7
5.1	Mapa de Influencia	7
5.2	Mapa de Tension	8
5.3	Mapa de Vulnerabilidad	9
6	Nivel Estratégico (IA Global)	10
6.1	Modo Ataque	10
6.2	Modo Defensa	11
6.3	Modo Vigilancia	11
6.4	Modo Guerra Total	11
7	Nivel táctico. IA individual	12
7.1	Selección de Roles	12
7.2	Máquinas de Estado y Flujo de Decisión	13
7.3	Patrulla y Detección de Enemigos	14
8	Pathfinding Táctico	15
8.1	Generación de malla y biomas	15
8.2	Influencia del mapa táctico con A*	16
8.3	Impacto en el comportamiento de la IA	16
9	Interfaz, Controles y Depuracion	17
9.1	Panel de Estado Táctico	17
9.2	Indicadores Visuales en el Entorno	18
9.3	Controles de Interacción y Depuración	19
10	Conclusión	20

1 Introducción

Esta memoria documenta el diseño y la implementación del **Bloque 2: IA Estratégica y Táctica** correspondiente a la práctica de la asignatura de Inteligencia Artificial para el Desarrollo de Juegos.

El objetivo principal de este proyecto es desarrollar un sistema de inteligencia artificial robusto dentro de un entorno de juego de guerra en tiempo real, desarrollado sobre el motor Unity 2023.2.20f1. A lo largo de este documento, se detallará los elementos implementados para gestionar la toma de decisiones algorítmicas en dos niveles diferenciados:

- El nivel *estratégico*, encargado de evaluar el estado global de la partida y dirigir el comportamiento de los bandos o facciones mediante distintos modos generales (ofensivo, defensivo y guerra total).
- El nivel *táctico*, responsable de las acciones individuales y especializadas de cada unidad en el campo de batalla.

Adicionalmente, se expondrá la integración de mecánicas de combate, el establecimiento de condiciones de victoria, y la generación de mapas tácticos. Siendo estos últimos elementos fundamentales, ya que su información se combina con el algoritmo de búsqueda A* para lograr un *pathfinding táctico*, permitiendo que las unidades no solo eviten obstáculos físicos, sino que también tomen rutas inteligentes basándose en el dominio territorial y la presencia enemiga.

2 Escenario

En este apartado se describe el mapa donde tiene lugar el enfrentamiento. A continuación detallaremos cómo está diseñado el terreno, dónde se ubican las zonas seguras y de reaparición para las tropas, y cuáles son los objetivos que debe cumplir cada bando para ganar la partida.

2.1 Descripción

En este escenario se enfrentan dos ejércitos autónomos (**Rojo** y **Azul**) en un mapa complejo, utilizando un río central y tres puentes como cuellos de botella naturales. Además, el entorno está dividido en diferentes biomas como praderas, zonas urbanas, bosques, caminos o ríos infranqueables. Cada bando dispone de su propia infraestructura logística (base, hospital, puntos de *respawn*, etc) y despliega unidades de cinco tipos distintos (Caballero, Arquero, Lancero, Tanque y Explorador). Estas unidades cuentan con comportamientos tácticos únicos y propiedades de combate específicas para cada bioma, haciendo que cada unidad reaccione y se posicione de manera completamente diferenciada ante los mismos estímulos.



Figura 1: Vista general del mapa del videojuego, mostrando los biomas, el río central y las bases.

2.2 Puntos seguros

Los puntos seguros son áreas estratégicas designadas en el mapa cuyo propósito principal es la recuperación de las tropas. Solo hay uno en cada equipo, cuando una unidad se encuentra dentro de una de estas zonas, se cura automáticamente, recuperando 30 puntos de vida por cada segundo que permanezca en ella. Esta mecánica de curación es universal, es decir, la tasa de regeneración es exactamente la misma para todos los NPCs, independientemente de su tipo o clase.



Figura 2: Zona de curación, se observa un círculo verde donde al entrar el NPC ya podrá empezar a curarse.

2.3 Respawn de los NPCs

Cuando un NPC muere, reaparece automáticamente pasado un pequeño tiempo de espera. El comportamiento es sencillo y pensado para mantener la batalla fluida: tras la muerte, la unidad vuelve a generarse en el punto de reaparición más cercano asignado a su equipo (existen 3 por cada equipo), siendo el tiempo de espera de **5 segundos**.

Al reaparecer, la unidad conserva su tipo (caballero, arquero, etc.) y su equipo, y recibe la orden de volver al lugar donde murió. Si el punto de spawn concreto ha sido destruido (se destruye si un NPC enemigo está 15 segundos en esa base), el sistema elige otro spawn disponible (o la base) para evitar que las unidades reaparezcan en un lugar no válido.



Figura 3: Vista del área de reaparición roja.



Figura 4: Vista del área de reaparición azul.

2.4 Objetivos

Los objetivos del mapa son los puntos que marcan la dirección principal de avance de cada bando. En nuestro proyecto, estos objetivos se usan como referencias estratégicas para que las unidades sepan hacia dónde dirigirse y qué zonas deben defender o atacar. Cada equipo tiene su propio castillo como objetivo principal y sus puntos asociados, que funcionan como parte importante del control del mapa.

Algunos puntos del mapa o objetivos secundarios pueden ser conquistados por el equipo enemigo. En concreto, las Torres (que son elementos de respawn), su captura se produce cuando un enemigo permanece dentro de la zona durante un tiempo continuo de **15 segundos**. Si durante la captura no hay enemigos en la zona o alguno recibe daño, el progreso se reinicia o se interrumpe.

Cuando un castillo cae (el objetivo principal), el juego detecta que ya se ha cumplido la condición de victoria. En ese momento se activa la pantalla del equipo ganador y la partida se detiene. Por tanto, la victoria no depende de una puntuación, sino de la destrucción del castillo enemigo. Además, los puntos de spawn asociados a torres o castillos pueden desactivarse cuando son destruidos o conquistados, para evitar que las unidades reaparezcan en zonas que ya no pertenecen al bando original. Esto hace que la conquista de objetivos afecte directamente al respawn y al control territorial.



Figura 5: Castillo principal del equipo rojo.

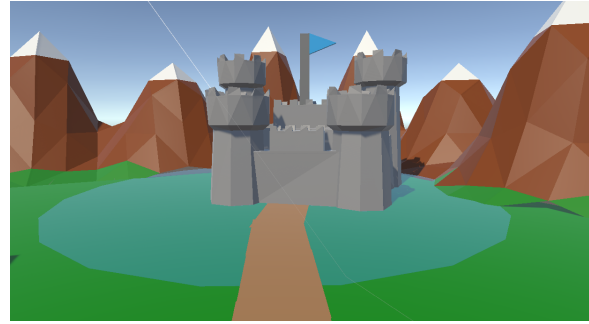


Figura 6: Castillo principal del equipo azul.

3 Sistema de Combate

El sistema de combate del proyecto no se basa en un cálculo plano de restas de vida, sino que implementa un modelo de resolución dinámico inspirado en los juegos de estrategia clásicos como *Age of Empires*. Este sistema evalúa los atributos base de las unidades, los cruza con matrices de ventajas por tipología y terreno, y aplica modificadores contextuales en tiempo de ejecución.

3.1 Tipo de Unidades y atributos

Como ya sabemos existen 5 tipos de unidades en el campo de batalla donde cada una de ellas cuenta con un componente `NPCStats` que almacena sus atributos. Para fomentar enfrentamientos tácticos duraderos y dar margen a la toma de decisiones como huir al hospital cuando la vida alcanza en 20 % de su máxima, se han configurado altos valores de salud y un sistema de poder. Este Poder o daño es un valor único que sirve tanto como fuerza de ataque como índice de defensa base.

Tabla 1: Estadísticas base por tipo de unidad

Unidad	Vida Max	Poder	Vel. Ataque
Tanque	1200	120	2.0 s
Caballero	1000	80	0.7 s
Lancero	900	90	1.2 s
Arquero	550	150	1.5 s
Explorador	650	120	0.8 s

3.1.1. Tabla de ventajas (FDA, FTA, FTD)

El script `SistemaCombate.cs` actúa como una clase de utilidad que almacena las tablas de resolución táctica (basadas en el Anexo II de los requisitos de la tarea). Con esto conseguimos realismo a la hora de poder equilibrar las unidades entre ellas y que tengan ventajas y desventajas según su terreno y su enemigo.

- **FAD (Factor Atacante/Defensor):** Determina la ventaja que tienen las unidades a la hora de atacar o recibir un golpe. Por ejemplo, los Lanceros tienen ventaja sobre un Caballero, pero sufren ante los Arqueros.

- **FTA (Factor de Terreno en Ataque):** Modifica el potencial ofensivo según el bioma desde el que se ataca. Por ejemplo, un Caballero tiene un aumento de potencia de un 50 % en la pradera en cambio pierde poder al estar en un bosque o un terreno urbano.
- **FTD (Factor de Terreno en Defensa):** Modifica el poder del defensor. Valores inferiores a 1.0 indican protección adicional frente al daño.

Tabla 2: FAD: Factor de ventaja por Tipos (Atacante \ Defensor)

	Arquero	Caballero	Explorador	Lancero	Tanque
Arquero	1.00	0.75	1.25	1.50	0.50
Caballero	1.50	1.00	0.75	0.50	1.25
Explorador	1.25	0.50	1.00	0.75	1.50
Lancero	0.50	1.50	1.25	1.00	0.75
Tanque	1.50	1.25	0.50	0.75	1.00

Tabla 3: FTA: Factor de Ataque por bioma

Bioma	Caballero	Arquero	Lancero	Tanque	Explorador
Pradera	1.50	1.00	1.00	1.25	1.00
Camino	1.25	1.00	1.00	0.75	0.75
Bosque	0.75	1.25	0.75	0.50	1.25
Urbano	0.50	0.75	1.25	1.50	1.00

Tabla 4: FTD: Factor de Defensa por bioma

Bioma	Caballero	Arquero	Lancero	Tanque	Explorador
Pradera	0.75	1.00	1.00	1.00	1.25
Camino	1.25	1.00	1.00	1.00	0.75
Bosque	1.25	0.75	1.25	1.25	1.00
Urbano	1.50	1.25	0.75	0.75	0.75

3.1.2. Daño crítico y bonus por situación

El sistema de daño y ataque está implementado mediante un modelo matemático de factores multiplicativos, diseñado para que el combate dependa del tipo de la unidad, terreno y el estado del NPC. La fuerza efectiva del atacante (FA) y del defensor (FD) se calcula cruzando sus atributos base con las tablas tácticas:

$$FA = \text{Poder}_{atacante} \cdot FAD \cdot FTA$$

$$FD = \text{Poder}_{defensor} \cdot FTD$$

El daño no es un número fijo, sino que se calcula dinámicamente con esta fórmula principal:

$$\text{Daño} = \left(\frac{FA}{FD} \right) \cdot \text{CTE_IMPACTO} \cdot \text{Aleatorio}[0,5, 1,0]$$

- **FA (Fuerza de Ataque):** Qué tan fuerte golpea el atacante.
- **FD (Fuerza de Defensa):** Qué tan bien se protege el defensor.
- **CTE_IMPACTO:** Una base de daño para que los combates duren un tiempo razonable.

A estos valores base hay que añadirles el uso de las tablas FAD, FTA y FTD, que multiplican la potencia de las unidades en función de dónde están y contra quién luchan.

Siempre hay un factor aleatorio entre el 50 % y 100 % del daño calculado para que los combates no sean deterministas y monotonos. Para darle aún más variabilidad al combate, existe una probabilidad del 1 % de asestar un golpe crítico, donde el daño será 50 veces la constante de impacto, capaz de eliminar a un NPC entero de un solo golpe.

Aunque el sistema de combate es igual para todos, cada NPC tiene comportamientos tácticos que afectan a sus estadísticas. Con esto hacemos que cada tropa sea diferente de otra del mismo tipo en el caso de que estén en dos estados distintos. Por ejemplo:

- **Lanceros:** Si están en estado de ataque y tienen aliados cerca, ganan un 20 % más de daño.
- **Tanques:** Si están solos (sin aliados cerca) defendiendo, ganan un 25 % más de poder.
- **Caballeros:** Si tienen menos del 50 % de vida y están atacando, entran en "furiaz" ganan un 20 % de daño.

3.2 Inicio de combate

Para empezar el combate el script `PercepcionNPC.cs` se encarga de definir un rango de percepcion para cada unidad. Todo enemigo que entre en este rango provocará que el estado del NPC transite inmediatamente a Combate. Esto significa que, si es una unidad cuerpo a cuerpo (*melee*), avanzará hacia el objetivo hasta alcanzar su rango de ataque mínimo; en el caso de un arquero, se detendrá y disparará a distancia. Los ataques se realizan siempre con la unidad estática (estando quietos).

Independientemente de si el NPC estaba patrullando o realizando una tarea distinta, su acción anterior se interrumpirá por completo. El combate se mantendrá activo de forma ininterrumpida hasta que ocurra una de estas tres condiciones: que el NPC necesite huir para curarse, que pierda de vista a su objetivo, o que uno de los dos combatientes muera.

Una vez en posición y ejecutando el ataque, el script invoca el método `CalcularDanio` para resolver el impacto. Esto resta la salud al enemigo y actualiza sus atributos en tiempo

real, un paso crítico ya que esta pérdida de vida puede forzar al objetivo a cambiar su comportamiento interno y priorizar una retirada táctica para curarse.

4 Condiciones de Victoria

Como se mencionó anteriormente la victoria se alcanza cuando cae el castillo enemigo; en ese momento se muestra la pantalla del bando ganador y la partida se detiene. Para conquistar el castillo las tropas deben permanecer durante 30 segundos en la región del castillo.

5 Mapas Tacticos

El sistema de mapas tácticos es uno de los componentes centrales de este proyecto, proporcionando una visualización dinámica del estado del combate de los bandos Rojo y Azul cada 0.5 segundos. Permite a los jugadores y al sistema de inteligencia artificial tomar decisiones estratégicas informadas basándose en la distribución de fuerzas en el campo de batalla.

El proyecto implementa tres mapas tácticos visuales localizada en la esquina de la pantalla durante el juego, permitiendo al jugador ciclar entre ellos mediante la tecla **M**.

5.1 Mapa de Influencia

El Mapa de Influencia proporciona una visualización clara del dominio territorial de cada bando en el campo de batalla. Su propósito es mostrar en todo momento qué zonas del mapa están bajo control del equipo Rojo y cuáles bajo control del equipo Azul. Cada unidad emite una influencia que se extiende radialmente desde su posición, decreciendo gradualmente con la distancia, marcando el territorio que el bando controla o tiene presencia activa. La influencia se calcula mediante la fórmula de decaimiento por distancia al cuadrado:

$$\text{Influencia} = \frac{\text{Fuerza}}{1 + d^2}$$

donde *Fuerza* es el poder de la unidad y *d* es la distancia en celdas desde la unidad. Esta fórmula asegura que cada unidad aporta máxima influencia en su posición pero decrece rápidamente con la distancia, permitiendo que el mapa refleje precisamente dónde están concentradas las fuerzas de cada bando.

Es importante recordar, que el mapa se actualiza cada 0.5 segundos conforme los movimientos y pérdidas de unidades afectan al control territorial. La visualización utiliza un código de colores intuitivo: las zonas dominadas por el equipo Rojo se representan en tonos rojos, mientras que las del equipo Azul en tonos azules, permitiendo identificar rápidamente las líneas de confrontación. Este tipo de mapas sirve de múltiples propósitos estratégicos: para el jugador, proporciona una vista aérea del estado de la batalla permitiéndole comprender el control territorial de cada equipo; para la IA, el sistema consulta constantemente este mapa para optimizar sus decisiones tácticas, evitando avanzar hacia territorios fuertemente dominados por el enemigo, reforzando zonas de control aliado débil,

y ajustando sus rutas de navegación para minimizar riesgo; y para la predicción estratégica, cambios rápidos en el Mapa de Influencia indican puntos de quiebre en las líneas de batalla, señalando dónde es probable que ocurran combates decisivos.

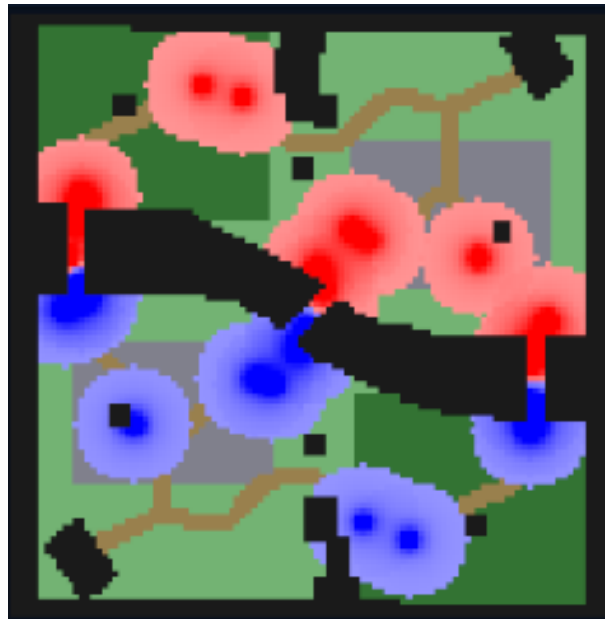


Figura 7: Mapa táctico de influencia, donde se observa como hay unidades atacando, defendiendo y vigilando en cada equipo.

5.2 Mapa de Tension

El Mapa de Tensión representa la actividad total del campo de batalla, combinando la influencia de ambos bandos para mostrar dónde se concentra la mayor cantidad de unidades y combate. Su propósito es identificar las zonas de máxima actividad y confrontación, permitiendo comprender en qué áreas del mapa hay mayor densidad de fuerzas sin importar de qué bando sean. La tensión se calcula como la suma total de las influencias de ambos bandos en cada celda:

$$\text{Tensión} = \text{InfluenciaRojo} + \text{InfluenciaAzul}$$

Donde obtenemos la concentración de unidades activas; picos altos en el mapa indican zonas donde ambos bandos tienen presencia significativa y por tanto donde la batalla es más intensa. El mapa se actualiza cada 0.5 segundos conforme las unidades se mueven y el número de combatientes cambia. La visualización utiliza un gradiente de colores que va desde tonos amarillos (baja tensión, pocas unidades presentes) hasta tonos naranja y rojo intenso (alta tensión, muchas unidades en combate activo), permitiendo al jugador identificar instantáneamente dónde está ocurriendo la acción principal.

El Mapa de Tensión sirve de múltiples propósitos estratégicos: para el jugador, proporciona una vista clara de dónde está concentrada la batalla y permite comprender el flujo general del combate sin preocuparse por bandos específicos; para la IA, el sistema consulta este mapa para detectar zonas de combate intenso y ajustar el refuerzo de unidades hacia puntos críticos, también permite identificar sectores del mapa abandonados donde es posible avanzar con menor resistencia; y para la toma de decisiones, picos repentinos

de tensión en nuevas áreas del mapa indican que la batalla se está expandiendo o que nuevos enfrentamientos están ocurriendo, señalando oportunidades tácticas o amenazas emergentes que requieren respuesta inmediata.



Figura 8: Mapa táctico de tensión, donde se observa los puntos calientes del escenario.

5.3 Mapa de Vulnerabilidad

El Mapa de Vulnerabilidad identifica las zonas disputadas del campo de batalla, mostrando dónde el poder está más equilibrado entre ambos bandos. Su propósito es revelar los frentes de batalla más críticos y vulnerables, donde ningún bando tiene dominancia clara y por tanto donde pequeños refuerzos o decisiones tácticas pueden cambiar el resultado del combate. La vulnerabilidad se calcula como la diferencia entre la tensión total y el valor absoluto de la influencia:

$$\text{Vulnerabilidad} = \text{Tensión} - |\text{Influencia}|$$

donde la Tensión es la suma de ambas influencias y la Influencia es la diferencia entre ellas. Esta fórmula revela que la vulnerabilidad es máxima cuando hay mucha actividad (tensión alta) pero ambos bandos están equilibrados (diferencia pequeña), y mínima cuando un bando domina claramente la zona sin competencia, se multiplica por un factor de 2 para intensificar. El mapa se actualiza cada 0.5 segundos conforme el equilibrio de fuerzas cambia en el campo de batalla. La visualización utiliza un gradiente de colores que va desde tonos blancos (baja vulnerabilidad, dominio claro de un bando) hasta tonos magenta intenso (alta vulnerabilidad, fuerzas equilibradas), permitiendo al jugador identificar instantáneamente las líneas de batalla más frágiles y disputadas.

El Mapa de Vulnerabilidad sirve de múltiples propósitos estratégicos: para el jugador, proporciona una vista de dónde el balance está más ajustado y donde una intervención táctica puede ser más decisiva; para la IA, el sistema consulta este mapa para concentrar recursos en zonas vulnerables donde obtener ventaja es posible, también detecta dónde su

bando está siendo presionado y requiere refuerzos urgentes; y para la predicción estratégica, las zonas de máxima vulnerabilidad representan potenciales puntos de ruptura en las líneas enemigas, donde un ataque podría colapsar la defensa o donde se corre el riesgo de que la defensa propia colapse si no se refuerza adecuadamente.



Figura 9: Mapa táctico de vulnerabilidad, donde se observa que en los puentes es donde se sitúa la mayor vulnerabilidad.

6 Nivel Estratégico (IA Global)

El nivel estratégico actúa como el núcleo de toma de decisiones del ejército. A diferencia de la IA individual, que gestiona el combate y la supervivencia de una sola unidad, la IA global evalúa y decide la postura general de todo el bando. Al iniciar la partida, el ejército adopta por defecto una postura defensiva.

A través del componente `TacticalCanvasController`, el sistema redistribuye dinámicamente los roles de las unidades. Para lograrlo, el controlador asigna numéricamente cuántos NPCs deben asumir los modos de Ataque, Defensa o Vigilancia, adaptando la composición del ejército según el modo estratégico seleccionado por el jugador.

6.1 Modo Ataque

Este modo adopta una configuración marcadamente agresiva. Para una población estándar de 15 unidades por bando, el sistema intentará repartirlas de la siguiente manera:

- **8 unidades en Ataque:** Priorizan buscar al enemigo, adentrarse en su territorio y avanzar hacia los objetivos principales.
- **4 unidades en Vigilancia:** Patrullan el mapa para proteger en puntos críticos del bando.
- **3 unidades en Defensa:** Se quedan atrás para proteger las infraestructuras clave del equipo.

Con esta distribución se fuerza al bando a tener una amplia mayoría de soldados con intenciones puramente ofensivas, trasladando la presión al territorio rival y generando un mayor número de conflictos directos para desgastar las líneas enemigas.

6.2 Modo Defensa

Este modo adopta una postura mucho más conservadora, diseñada para resistir al rival. Para el mismo conjunto de 15 unidades, la distribución se invierte:

- **3 unidades en Ataque:** Realizan pequeñas incursiones al territorio enemigo para debilitar al rival.
- **6 unidades en Vigilancia:** Mantienen un control extenso de las rutas.
- **6 unidades en Defensa:** Se atrincheran en las zonas seguras aliadas.

Esta configuración evita el malgasto innecesario de tropas en ofensivas arriesgadas. En su lugar, busca dominar el espacio propio y favorecer que las unidades aprovechen al máximo sus comportamientos tácticos defensivos individuales.

6.3 Modo Vigilancia

Más que una directiva global excluyente, la vigilancia actúa como la capa de mantenimiento del mapa. Su objetivo no es empujar agresivamente la línea de frente ni atrincherarse estáticamente en la base, sino mantener un control pasivo del terreno, recabar información táctica y asegurar que las rutas estén libres de enemigos.

Las unidades bajo esta cuota recorren sus caminos de *waypoints*, manteniendo un bajo coste computacional mientras proyectan la influencia de su bando por las zonas neutrales del escenario. Durante estas patrullas, los agentes continúan evaluando los costes de los terrenos en su navegación A^* y, si detectan un objetivo rival, interrumpirán la ruta para neutralizarlo.

6.4 Modo Guerra Total

Esta es una orden definitiva diseñada exclusivamente para forzar la condición de victoria: destruir el castillo enemigo. Al activarse, se anulan todos los filtros tácticos de precaución ante rivales y se fuerza un asalto masivo coordinado. Para garantizar la efectividad de la orden, el sistema aplica dos alteraciones radicales en la lógica de las unidades:

1. **Sobrescritura de Roles (`estadoNPC.cs`):** La directiva ignora cualquier distribución previa de Ataque, Defensa o Vigilancia. El script guarda en memoria qué tarea estaba realizando cada unidad y automáticamente fuerza a casi todas a adoptar el modo `.Ataque`. Si la Guerra Total se desactiva (tecla T), todas las tropas recuperan instantáneamente su trabajo y directiva original.
2. **Reasignación de Objetivos (`PercepcionNPC.cs`):** Las unidades dejan de pelear por el control del mapa, ignoran los objetivos intermedios y centran su navegación directamente en la condición de victoria. El comportamiento durante este asedio se divide estrictamente por tipo de tropa:

- **Vanguardia (Tanques, Caballeros y Exploradores):** Ejecutan un asalto directo, marchando en línea recta para sitiar y destruir el castillo enemigo sin distracciones.
- **Apoyo de Asedio (Arqueros):** Avanzan hacia el castillo rival, pero calculan una posición estratégica a 8 unidades métricas de distancia, garantizando la aplicación de daño constante desde fuera del alcance del combate cuerpo a cuerpo.
- **Defensores de Base (Lanceros):** Actúan como el seguro del bando frente a incursiones por la espalda (*base race*). Se retiran inmediatamente a su propio castillo y forman una línea defensiva a 4 unidades métricas por delante del mismo, con la única misión de interceptar a los atacantes enemigos.

Finalmente, la IA está programada para reconocer el éxito de la campaña: si el castillo enemigo cae, las tropas asaltantes detendrán automáticamente su avance, dando por concluida la contienda.

7 Nivel táctico. IA individual

7.1 Selección de Roles

En el nivel táctico individual, la Inteligencia Artificial no se limita a empujar a las unidades hacia el enemigo más cercano. Cuando el controlador estratégico global emite una orden como pasar a estado Ataque, el agente interpreta esa orden basándose en su tipo de Unidad y en el análisis de su entorno inmediato.

El sistema evalúa constantemente el componente **NPCStats** y la posición de los rivales (mediante la tabla táctica de *FA/FD*) para decidir a quién atacar y dónde posicionarse. Este sistema de selección de roles dota de personalidad a cada NPC, evitando comportamientos genéricos y generando en su lugar los siguientes comportamientos tácticas según la unidad:

- **Caballero:**
 - **En Ataque:** Busca los puntos de mayor influencia enemiga para frenar su paso. Si su salud baja del 50 %, entra en estado de furia, aumentando su poder en un 20 %.
 - **En Defensa:** Se posiciona en la frontera táctica (donde la influencia aliada y enemiga empatan). Si recibe daño y está aislado, se retira hacia la posición de sus aliados más cercanos.
- **Arquero:**
 - **En Ataque:** Busca zonas seguras dominadas por la influencia aliada para proporcionar ataque desde la retaguardia.
 - **En Defensa:** si un enemigo se acerca demasiado, interrumpe su ataque y retrocede para mantener una distancia segura. Cuando no está en combate, evalúa el mapa de influencia para posicionarse en zonas con baja densidad aliada pero con visión al avance enemigo, garantizando líneas de tiro limpias en los flancos sin perder la proximidad a la torre que defiende.

- **Lancero:**

- **En Ataque:** Avanza en bloque. Gana un bono del +20 % de daño siempre que tenga aliados cerca, incentivando el avance grupal.
- **En Defensa:** Al permanecer parado defendiendo, su capacidad de mitigación de daño aumenta un 25 %.

- **Tanque:**

- **En Ataque:** Funciona como unidad de asedio. Su directiva le impide retirarse a curar; su único objetivo es avanzar hacia las torres o el castillo rival para absorber daño.
- **En Defensa:** Especialista en retener puntos críticos. Si defiende una posición en solitario (sin apoyo aliado), su poder aumenta un 25 %.

- **Explorador:**

- **En Ataque:** Opera como unidad asesina. Ignora a los objetivos más cercanos y prioriza dar caza a las unidades enemigas con menor porcentaje de salud.
- **En Defensa:** Su prioridad es la supervivencia y patrullar donde hay poca influencia. Si entra en combate, huye inmediatamente hacia el Bosque más cercano para beneficiarse de su bono de terreno.

7.2 Máquinas de Estado y Flujo de Decisión

Cada NPC mantiene una Máquina de Estados a través del script `estadoNPC.cs`, la cual actúa como núcleo. Esta máquina permite a la unidad conocer en qué contexto se encuentra en cada frame, posibilitando una actuación táctica.

La máquina define tres estados globales, los cuales son fácilmente identificables por el jugador mediante un holograma proyectado sobre el NPC:

- **Vigilancia (Icono de Ojo):** El agente adopta un rol donde sigue una serie de puntos y patrulla la base aliada.
- **Ataque (Icono de Espada):** El agente entra en rutinas ofensivas.
- **Defensa (Icono de Escudo):** El agente se ancla en su posición para proteger el territorio.

Sin embargo, el `estadoNPC.cs` funciona únicamente como el cerebro estratégico que dicta el estado de la unidad. El comportamiento táctico real ocurre cuando ese modo se combina con el tipo de unidad específica. Este proceso interno se ejecuta paso a paso a través del siguiente flujo de decisiones:

1. **El Selector de Modo (`estadoNPC.cs`):** Su función principal es transitar entre los estados y notificar al resto de componentes. Por ejemplo, al entrar en estado defensivo, invoca métodos como `percepcion.PrepararDefensa()`.

2. **El Filtro Táctico (PercepcionNPC.cs):** Dentro del bucle de actualización (`Update`), el NPC consulta constantemente su estado. Mediante condicionales lógicos, el flujo de ejecución varía según los estados concretos como `GestionarDefensa()` o `GestionarAtaque()`.
3. **La Diferenciación por Unidad:** Dentro de las rutinas de gestión ocurre la verdadera especialización táctica. El código evalúa mediante una estructura `switch` la clase del NPC (Caballero, Lancero, Arquero...). Es aquí donde se decide que un Explorador en Defensa activa la huida hacia los bosques, o que un Caballero busque la línea de frente.
4. **Modificadores de Combate (SistemaCombate.cs):** La máquina de estados también afecta directamente a las estadísticas en tiempo real. El script de combate consulta la FSM para aplicar bonificaciones; por ejemplo, si un Lancero está en Defensa y su velocidad es 0, gana un +25 % extra de protección.

En resumen, la Máquina de Estados establece el contexto global de la unidad, y los scripts de Percepción y Combate utilizan ese contexto, junto con el Mapa de Influencia, para ejecutar un comportamiento especializado.

7.3 Patrulla y Detección de Enemigos

El ciclo de vida básico de una unidad se divide entre el recorrido continuo de un circuito cerrado (Patrulla) y la monitorización constante de su entorno (Detección). La detección de amenazas es el evento de mayor prioridad en el árbol de decisiones de la IA y detendrá cualquier rutina actual para que el NPC se acerque e inicie el combate. Existe una única excepción a esta regla: **la intervención directa del jugador**. Si el usuario selecciona manualmente a una unidad y le ordena moverse a unas coordenadas específicas, la directiva manual actúa como un *override* absoluto; el NPC marchará directo hacia su destino ignorando cualquier presencia enemiga en su radio.

Cuando el jugador no interviene, las unidades operan de forma autónoma gestionadas por los siguientes subsistemas:

- **Sistema de Patrulla (NPCPatrol.cs):**
 - **Circuito por Bandos:** Al inicializarse, el NPC lee la lista de *waypoints* (`waypointsRojo` o `waypointsAzul`) correspondiente a su bando, estableciendo un circuito de vigilancia.
 - **Navegación Inteligente:** A diferencia de un movimiento simple en línea recta, el agente calcula rutas completas mediante A*. Esto garantiza que el NPC rodeará zonas de alto coste para la unidad.
 - **Interrupción por Estado:** La patrulla funciona exclusivamente en el estado de Vigilancia. Si la máquina de estados dicta un cambio a Ataque o Defensa, el componente de patrulla se desactiva, cediendo el control de movimiento a la lógica del nuevo estado.
- **Sistema de Detección (PercepcionNPC.cs):**

- **Escaneo por Intervalos:** Para no saturar la CPU calculando distancias en cada *frame*, el NPC no evalúa su entorno de forma continua. Utiliza un `Physics.OverlapSphere` optimizado que se ejecuta cada `intervaloBusqueda` de 0.2 segundos.
- **Radio Dinámico:** El tamaño del área de percepción varía según el contexto. Si el NPC entra en estado de Ataque, su radio matemático se multiplica por 1,6, simulando un estado de alerta que le permite localizar objetivos desde mayor distancia.
- **Priorización de Objetivos (Targeting):** Una vez detectados múltiples enemigos, la lógica general ordena fijar al más cercano para interceptarlo. Sin embargo, la modularidad del código permite excepciones: por ejemplo, el Explorador en modo Ataque ignora la proximidad y filtra a los enemigos de su radio para lanzarse contra aquel que tenga menos salud.

8 Pathfinding Táctico

8.1 Generación de malla y biomas

Como ya sabemos el escenario cuenta con un río central y tres puentes que actúan como cuellos de botella para forzar los encuentros tácticos entre ambos bandos. El entorno virtual se divide mediante el script `GridManager.cs`, el cual genera una cuadrícula bidimensional de 105×105 celdas. Cada una de estas celdas cuenta con un tamaño físico de 3×3 unidades para asegurar un mapeo preciso del terreno sin perder precisión en los límites de cada bioma.

Durante la inicialización, el sistema recorre el mundo calculando el centro de cada nodo y realiza el siguiente proceso para construir el grafo:

- **Cálculo de transitabilidad:** Se verifica si la celda es transitable a través de la etiqueta *Obstaculo*. Si detecta colisión, la celda queda bloqueada para el paso como en el caso de los ríos y las montañas.
- **Asignación de Biomas:** Se determina a cuál de los cuatro biomas transitables pertenece la celda (Pradera, Camino, Bosque, Urbano) mediante un sistema de dos fases:
 1. Se comprueba si la celda colisiona con algún collider definido en `biomasColliders` como son los caminos del mapa. Esto permite mapear terrenos con formas irregulares y complejas.
 2. Si no se detecta colisión, se comprueba matemáticamente si las coordenadas caen dentro de alguna de las `biomasAreas` que son Bosque y Urbano. Esta evaluación de coordenadas es mucho más barata computacionalmente y se usa para cubrir extensiones masivas de terreno de forma eficiente.
 3. Por defecto, si un nodo transitable no pertenece a ninguna de estas clasificaciones, se asume el bioma de *Pradera*.
- **Mapeo de Puntos Estratégicos:** Integrado en este espacio delimitado por el *grid* o la malla, el módulo `WayPoints.cs` registra las ubicaciones exactas de las infraestructuras de ambos bandos (Hospitales, Castillos y puntos de reaparición)

para proporcionar objetivos de navegación válidos a la inteligencia artificial de cada NPC.

8.2 Influencia del mapa táctico con A*

El sistema de navegación estándar de un videojuego suele limitarse a encontrar la ruta más corta o la que menor coste de terreno supone. Sin embargo, en un entorno bélico, el camino más corto a través de un grupo de enemigos rara vez es el más ventajoso. Para solucionar esto, el algoritmo A* implementado incorpora una capa de **Pathfinding Táctico**.

$$A^* \rightarrow f(n) = g(n) + h(n)$$

Este componente dinámico puede alternarse en tiempo real mediante la tecla **P**. Cuando esta modalidad está encendida, el algoritmo altera la forma en la que calcula el coste real acumulado (g) para llegar a cada nodo vecino. Específicamente, cuando el Pathfinding Táctico está activo, se suma una **penalización táctica** al coste de movimiento de cada paso. Esta penalización se calcula a partir del **mapa de influencia**: por cada nodo, el algoritmo consulta cuánta influencia enemiga hay en esa posición, y ese valor se multiplica por un factor de 2,5 antes de agregarlo al coste total. La fórmula que se aplica es:

$$g_{\text{nuevo}} = g_{\text{actual}} + \text{coste del paso} + \text{penalización de terreno} + (\text{influencia enemiga} \times 2,5)$$

De esta forma, los NPCs evitan deliberadamente las zonas donde hay mayor concentración enemiga, eligiendo caminos tácticos más seguros incluso si son más largos. El factor multiplicador de 2,5 asegura que el desvío sea significativo y el comportamiento visiblemente táctico.

8.3 Impacto en el comportamiento de la IA

Al sumar directamente esta penalización táctica a la variable g del A*, el algoritmo percibe las zonas dominadas por el rival como caras en cuanto a coste de transitarlas.

El resultado es que la IA prefiere asumir el coste de dar un rodeo importante incluso si esto implica atravesar un bioma que penalice su movimiento, como un bosque o un área urbana antes que cruzar una zona con alta tensión enemiga. Matemáticamente, el coste acumulado del rodeo resulta más barato que el coste de atravesar la influencia enemiga, logrando que los agentes eviten a rivales, eviten bases fuertemente defendidas y maximicen su supervivencia.



(a) Pathfinding Estándar



(b) Pathfinding Táctico

Figura 10: Comparativa de la misma orden de movimiento. En (a), el A* calcula la ruta ignorando el peligro; en (b), la penalización por influencia enemiga encarece la ruta central obligando a la unidad a dar un rodeo estratégico.

9 Interfaz, Controles y Depuracion

Para que la ejecucion de la escena resulte comprensible, el proyecto integra una Interfaz de Usuario estructurada en varios niveles y un completo mapeo de controles que permite tanto la jugabilidad como la depuración en tiempo real.

9.1 Panel de Estado Táctico

En la pantalla, el usuario dispone de un Panel Táctico o HUD que proporciona información global de los bandos:

- **Estado Global:** Muestra de forma constante la directiva actual del ejército (Ataque, Defensa o Guerra Total) y si el *Pathfinding Táctico* está activado o no.
- **Minimapa Dinámico:** Un panel en una esquina que proyecta los mapas matemáticos generados por *MapaInfluencia.cs*. Permite al jugador ver en tiempo real el control territorial.

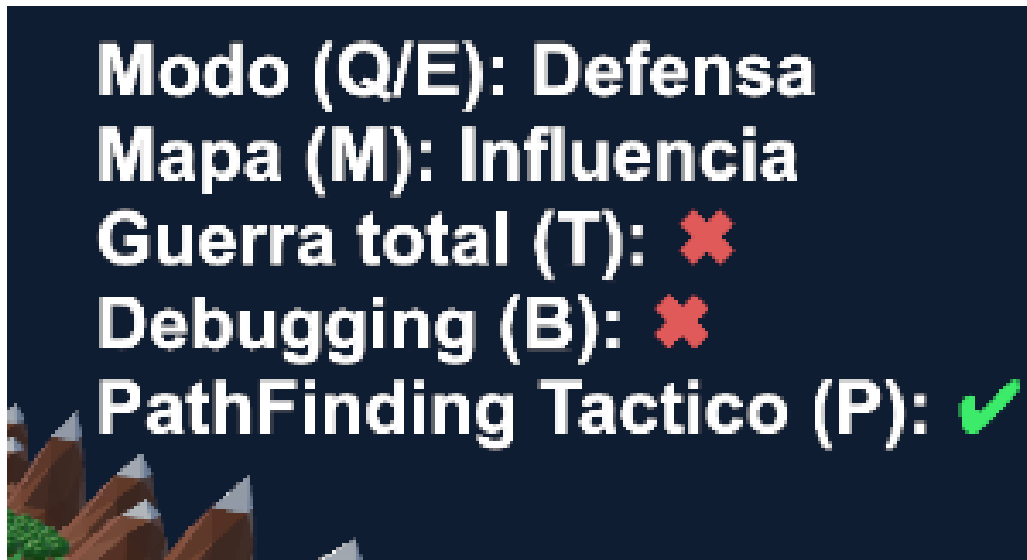


Figura 11: HUD donde se visualizan las opciones globales activadas y desactivadas.

9.2 Indicadores Visuales en el Entorno

Para facilitar la comprensión de la batalla, las unidades proyectan información sobre sí mismas:

- **Barras de Salud:** Cada NPC cuenta con una barra de vida sobre su modelo que cambia de color dinámicamente al descender, alertando visualmente de cuándo se acerca al umbral del 20 % para iniciar su huida al hospital.
- **Estado:** Sobre la cabeza de las unidades se renderiza una espada en caso de estar en Ataque, escudo si esta en Defensa y un Ojo en el caso de que este vigilando, permitiendo identificar de un vistazo cómo se está comportando cada unidad.

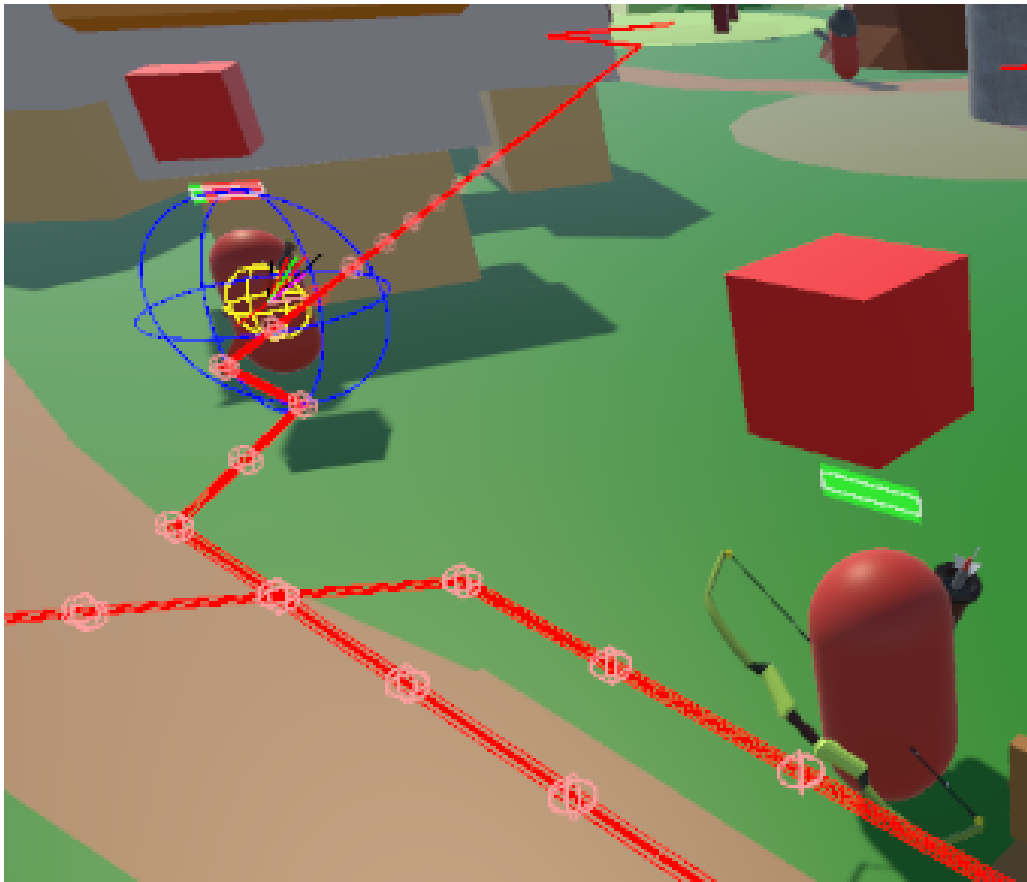


Figura 12: Indicadores visuales en el entorno: barras de vida y hologramas de estado sobre los NPCs.

9.3 Controles de Interacción y Depuración

Para verificar el correcto funcionamiento matemático del sistema (el algoritmo A*, las curaciones y los cambios de estado), se ha mapeado una serie de comandos de teclado de uso exclusivo para el desarrollo y depuración táctica:

Tabla 5: Mapa de teclas para interacción y depuración

Comando	Funcionalidad
Click Izquierdo	Selecciona un NPC o le ordena moverse manualmente a un punto.
Q / E	Alterna el Modo de combate global (Defensa / Ataque).
T	Activa / Desactiva la directiva extrema de Guerra Total.
M	Cicla la vista del Minimapa (Influencia → Vulnerabilidad → Tensión).
P	Alterna dinámicamente el motor de Pathfinding Táctico en los NPCs.
B	Modo Depuración: Muestra visualmente la cuadrícula (<i>grid</i>), los colores de los biomas, los radios de percepción esféricos y traza líneas renderizando las rutas A* activas de cada unidad.
J / K	Depuración de Daño: Inflige una cantidad de daño fijo (J) o cura (K) directamente a la unidad seleccionada, útil para forzar transiciones en la máquina de estados sin esperar al combate.

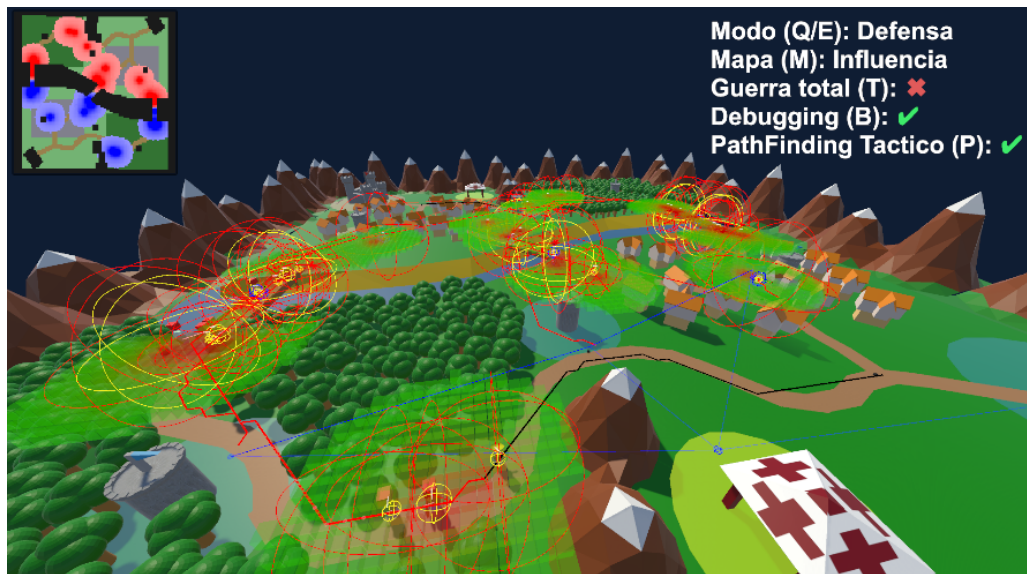


Figura 13: Vista del Modo Depuración (activado con la tecla B), mostrando la cuadrícula geométrica y las rutas de navegación trazadas por el algoritmo A*.

10 Conclusión

El desarrollo del Bloque 2 ha logrado integrar con éxito una arquitectura de Inteligencia Artificial de múltiples capas en el motor Unity. La separación modular en componentes específicos (*SistemaCombate*, *MapaInfluencia*, *EstadoTacticoGlobal*) ha demostrado ser una decisión de diseño altamente efectiva. Esta arquitectura ha permitido que las directivas estratégicas de alto nivel (como los Modos Ofensivo, Defensivo y la Guerra Total) fluyan de manera orgánica hacia el comportamiento táctico individual de cada clase, facilitando la ejecución de estrategias tácticas de las unidades y las retiradas tácticas de

supervivencia.

Uno de los mayores logros del proyecto ha sido la consecución de un comportamiento emergente realista. Gracias a la combinación de las matrices matemáticas de combate (FAD, FTA, FTD), el daño no determinista y el Pathfinding Táctico, las batallas resultan dinámicas e impredecibles. El uso de los Mapas Tácticos (Influencia, Tensión y Vulnerabilidad) dota a las unidades de una auténtica conciencia espacial, logrando que los agentes tomen la iniciativa de evitar zonas de alta concentración de unidades.